

DAY 6 TASKS

Vagdevi Malineni

malinenv24@gmail.com

Task 1 — User Authentication System

Step 1 — Create. Env

```
.env
1  PORT=5000
2  MONGO_URI=mongodb://127.0.0.1:27017/authDB
3  JWT_SECRET=mysecretkey
```

Step 2 — Database Connection

```
1  const mongoose = require("mongoose");
2
3  const connectDB = async () => {
4    try {
5      await mongoose.connect(process.env.MONGO_URI);
6
7      console.log("MongoDB Connected");
8    } catch (error) {
9      console.log(error.message);
10     process.exit(1);
11   }
12 };
13
14 module.exports = connectDB;
```

Step 3 — User Model

```
1  const mongoose = require("mongoose");
2  ....
3  const userSchema = new mongoose.Schema(
4    {
5      name: {
6        type: String,
7        required: true,
8      },
9
10     email: {
11       type: String,
12       required: true,
13       unique: true,
14     },
15
16     password: {
17       type: String,
18       required: true,
19     },
20   },
```

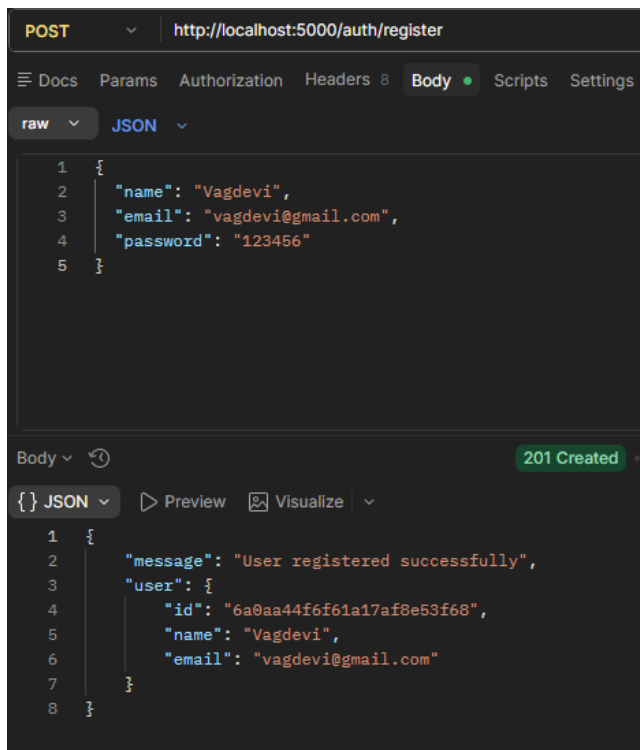
Step 4 — Main Server

```
1  const express = require("express");
2  const dotenv = require("dotenv");
3  const cors = require("cors");
4
5  const connectDB = require("../config/db");
6
7  dotenv.config();
8
9  connectDB();
10
11 const app = express();
12
13 app.use(cors());
14 app.use(express.json());
15
16 app.use("/auth", require("../routes/authRoutes"));
17
18 const PORT = process.env.PORT || 5000;
19
20 app.listen(PORT, () => {
```

Step 5 — Run Server

```
Node.js v24.15.0
PS C:\Users\V KRISHNA\auth-system> node server.js
◇ injected env (3) from .env // tip: override existing
  { override: true }
Server running on port 5000
MongoDB Connected
█
```

Step 6 — Test Register API



The screenshot displays a REST client interface with a POST request to `http://localhost:5000/auth/register`. The request body is a JSON object with the following fields:

```
{
  "name": "Vagdevi",
  "email": "vagdevi@gmail.com",
  "password": "123456"
}
```

The response status is `201 Created`. The response body is a JSON object:

```
{
  "message": "User registered successfully",
  "user": {
    "id": "6a0aa44f6f61a17af8e53f68",
    "name": "Vagdevi",
    "email": "vagdevi@gmail.com"
  }
}
```

Step 7 — Test Login API

```
POST http://localhost:5000/auth/login

{
  "email": "vagdevi@gmail.com",
  "password": "123456"
}

200 OK • 128 ms • 567 B
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZhMGFhNDRmNmY2MWEwN2FmOGU1M2Y2OCIsIm1hdCI6MTc3OTA4MzAzMiwiZXhwIjoxNzcsMTY5NDMyfQ.q-N4S7nssfYy7x2bG9m4mv7aH-a8HjZvsFvzg0XaAio",
  "user": {
    "id": "6a0aa44f6f61a17af8e53f68",
    "name": "Vagdevi",
    "email": "vagdevi@gmail.com"
  }
}
```

Task 2 — Protected Route Implementation

Step 1 — Create Middleware

```
middleware > JS authMiddleware.js > ...

1 const jwt = require("jsonwebtoken");
2
3 const authMiddleware = async (req, res, next) => {
4   try {
5
6     // Get token from header
7     const token = req.header("Authorization");
8
9     // Check token exists
10    if (!token) {
11      return res.status(401).json({
12        message: "Access denied. No token provided",
13      });
14    }
15
16    // Verify token
17    const decoded = jwt.verify(token, process.env.JWT_SECRET);
18
19    // Save user data in request
20    req.user = decoded;
```

Step 2 — Login First

```
{ } JSON Preview Visualize
1 {
2   "message": "Login successful",
3   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZhMGFhNDRmNmY2MWExN2FmOGU1M2Y2OCIsIm1hdCI6MTc3OTA4MzM4MiwiZXhwIjoxNzc5MTY5NzgyfQ.1bvFx7JhbpB7KeMs4hSaGuuDPGP5HwjK3nLkzYJL568",
4   "user": {
5     "id": "6a0aa44f6f61a17af8e53f68",
6     "name": "Vagdevi",
7     "email": "vagdevi@gmail.com"
8   }
9 }
```

Step 3 — Test Protected Route

```
GET http://localhost:5000/auth/profile
Docs Params Authorization Headers 9 Body Scripts Settings
Postman-Token <calculated when request is sent>
Content-Type application/json
Content-Length <calculated when request is sent>
Host <calculated when request is sent>
User-Agent PostmanRuntime/7.54.0
Accept */*
Accept-Encoding gzip, deflate, br
Connection keep-alive
Authorization Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZhMGFhNDRmNmY2MWExN2FmOGU1M2Y2OCIsIm1hdCI6MTc3OTA4MzM4MiwiZXhwIjoxNzc5MTY5NzgyfQ.1bvFx7JhbpB7KeMs4hSaGuuDPGP5HwjK3nLkzYJL568
Key Value Description
Body 200 OK 36 ms 372 B
{ } JSON Preview Visualize
1 {
2   "message": "Authorized user",
3   "user": {
4     "id": "6a0aa44f6f61a17af8e53f68",
5     "iat": 1779083382,
6     "exp": 1779169782
7   }
8 }
```

Task 3 — CRUD API Enhancement

Step 1 — Create Product Model

```

models > JS Product.js > ...
1  const mongoose = require("mongoose");
2
3  const productSchema = new mongoose.Schema(
4  {
5      productName: {
6          type: String,
7          required: true,
8          trim: true,
9      },
10
11      price: {
12          type: Number,
13          required: true,
14      },
15
16      brand: {
17          type: String,
18          required: true,
19          trim: true,
20  },

```

Step 2 — Create Product Controller

```

controllers > JS productController.js > ...
1  const Product = require("../models/Product");
2
3
4  // CREATE PRODUCT
5  const createProduct = async (req, res) => {
6      try {
7
8          const {
9              productName,
10             price,
11             brand,
12             category,
13             stock,
14         } = req.body;
15
16         // Validation
17         if (
18             !productName ||
19             !price ||
20             !brand ||

```

Step 3 — Create Routes

```
routes > JS productRoutes.js > ...
1  const express = require("express");
2
3  const {
4    createProduct,
5    getProducts,
6    updateProduct,
7    deleteProduct,
8  } = require("../controllers/productController");
9
10 const router = express.Router();
11
12
13 // CREATE
14 router.post("/", createProduct);
15
16
17 // GET
18 router.get("/", getProducts);
19
20
```

API Testing

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:5000/api/products
- Body (raw JSON):**

```
{
  "productName": "iPhone 15",
  "price": 80000,
  "brand": "Apple",
  "category": "Mobile",
  "stock": 10
}
```
- Status:** 201 Created
- Response (JSON):**

```
{
  "success": true,
  "message": "Product created successfully",
  "data": {
    "productName": "iPhone 15",
    "price": 80000,
    "brand": "Apple",
    "category": "Mobile",
    "stock": 10,
    "_id": "6a0aabf1a4fb5332f2bace28",
    "createdAt": "2026-05-18T06:04:33.756Z",
    "updatedAt": "2026-05-18T06:04:33.756Z",
    "__v": 0
  }
}
```

```
GET http://localhost:5000/api/products

Docs Params Authorization Headers 9 Body • Scripts Settings

Body 200 OK

JSON Preview Visualize

1 {
2   "success": true,
3   "count": 1,
4   "data": [
5     {
6       "_id": "6a0aabf1a4fb5332f2bace28",
7       "productName": "iPhone 15",
8       "price": 80000,
9       "brand": "Apple",
10      "category": "Mobile",
11      "stock": 10,
12      "createdAt": "2026-05-18T06:04:33.756Z",
13      "updatedAt": "2026-05-18T06:04:33.756Z",
14      "__v": 0
15    }
16  ]
17 }
```

```
PUT http://localhost:5000/api/products/6a0aabf1a4fb5332f2bace28

Docs Params Authorization Headers 9 Body Scripts Settings
raw JSON

1 {
2   | "price": 75000
3   | }

Body
{} JSON Preview Visualize

1 {
2   | "success": true,
3   | "message": "Product updated successfully",
4   | "data": {
5   |   | "_id": "6a0aabf1a4fb5332f2bace28",
6   |   | "productName": "iPhone 15",
7   |   | "price": 75000,
8   |   | "brand": "Apple",
9   |   | "category": "Mobile",
10  |   | "stock": 10,
11  |   | "createdAt": "2026-05-18T06:04:33.756Z",
12  |   | "updatedAt": "2026-05-18T06:08:37.624Z",
13  |   | "__v": 0
14  |   | }
15  | }
```

```
DELETE http://localhost:5000/api/products/6a0aabf1a4fb5332f2bace28

Docs Params Authorization Headers 9 Body Scripts Settings
raw JSON

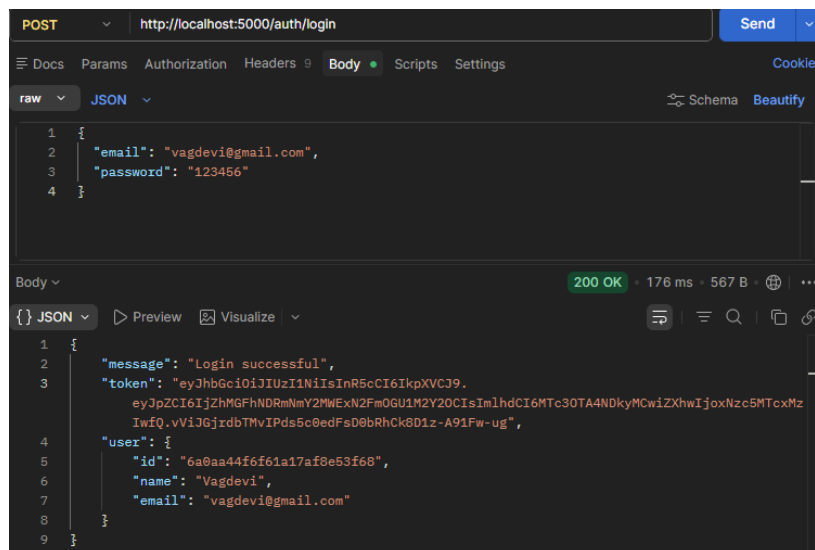
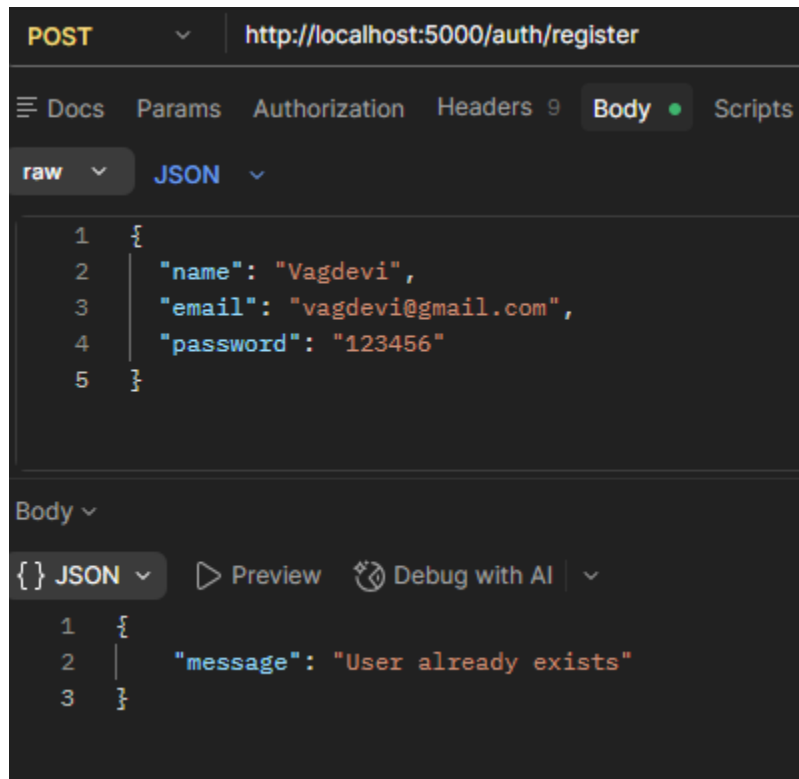
1 {
2   | "price": 75000
3   | }

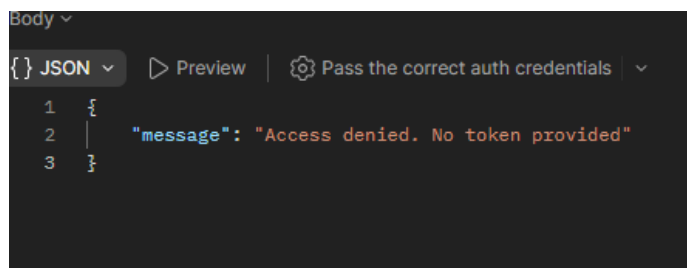
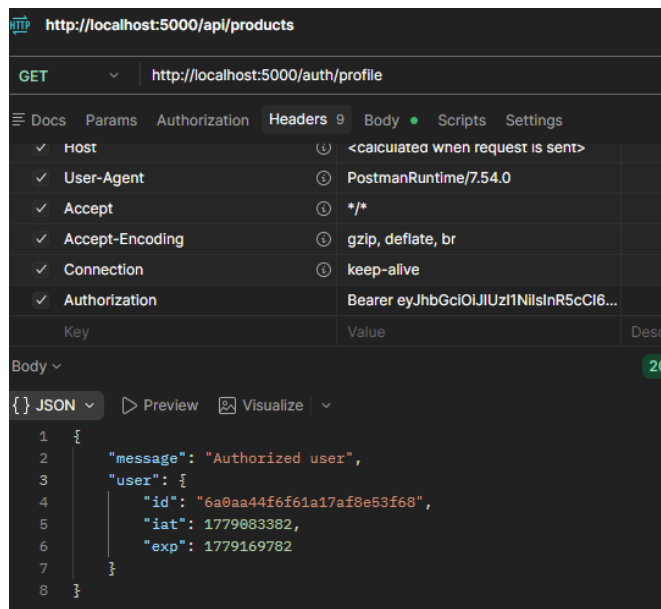
Body
{} JSON Preview Visualize

1 {
2   | "success": true,
3   | "message": "Product deleted successfully"
4   | }
```

Task 4 — API Testing using Postman

Authentication APIs





CRUD APIs

```
1 {
2   "success": true,
3   "message": "Product created successfully",
4   "data": {
5     "productName": "iPhone 15",
6     "price": 80000,
7     "brand": "Apple",
8     "category": "Mobile",
9     "stock": 10,
10    "_id": "6a0aaf23a4fb5332f2bace34",
11    "createdAt": "2026-05-18T06:18:11.002Z",
12    "updatedAt": "2026-05-18T06:18:11.002Z",
13    "__v": 0
14  }
15 }
```

```
GET http://localhost:5000/api/products

Docs Params Authorization Headers 8 Body Scripts Settings

raw JSON

1 {
2   "productName": "iPhone 15",
3   "price": 80000,
4   "brand": "Apple",
5   "category": "Mobile",
6   "stock": 10
7 }
```

Body

```
{
  "success": true,
  "count": 1,
  "data": [
    {
      "_id": "6a0aaf23a4fb5332f2bace34",
      "productName": "iPhone 15",
      "price": 80000,
      "brand": "Apple",
      "category": "Mobile",
      "stock": 10,
      "createdAt": "2026-05-18T06:18:11.002Z",
      "updatedAt": "2026-05-18T06:18:11.002Z",
      "__v": 0
    }
  ]
}
```

```
PUT http://localhost:5000/api/products/6a0aaf23a4fb5332f2bace34

Docs Params Authorization Headers 8 Body Scripts Settings

raw JSON

1 {
2   "productName": "iPhone 15",
3   "price": 80000,
4   "brand": "Apple",
5   "category": "Mobile",
6   "stock": 10
7 }
```

Body

```
{
  "success": true,
  "message": "Product updated successfully",
  "data": {
    "_id": "6a0aaf23a4fb5332f2bace34",
    "productName": "iPhone 15",
    "price": 80000,
    "brand": "Apple",
    "category": "Mobile",
    "stock": 10,
    "createdAt": "2026-05-18T06:18:11.002Z",
    "updatedAt": "2026-05-18T06:19:27.170Z",
    "__v": 0
  }
}
```

